

# AgroXAI Backend Architecture & Prediction Pipeline

A complete guide to the intelligence engine powering AgroXAI.

## 1. High-Level Architecture

AgroXAI operates on a robust microservices-inspired architecture spanning three primary environments:

- **Frontend:** React (Vite) / Tailwind CSS
- **API Gateway & Services:** Node.js / Express
- **Machine Learning Engine:** Python / Flask / XGBoost

### Data Flow Overview

1. **User Request:** The React frontend sends a minimal payload (location or manual parameters).
2. **Data Enrichment:** The Node.js backend acts as an orchestrator, aggressively gathering multi-dimensional environmental data (geospatial coordinates, 14-day climate averages, soil nitrogen/pH levels) using cached external services.
3. **ML Inference:** The enriched data is sent to the Python Flask server where complex feature engineering occurs, followed by XGBoost prediction and Agricultural Compatibility Index (ACI) scoring.
4. **Final Response:** Node.js filters the ML output to ensure real-time climate safety, packages it with explainability metrics, and returns it to the user.

## 2. The Prediction Pipeline (Step-by-Step)

### Step 1: Frontend Trigger

The user requests a recommendation in the UI (`Recommend.jsx`). The frontend issues a `POST /api/recommend` to the Node.js backend.

### Step 2: Node.js Service Orchestration (`recommend.js` & `agroServices.js`)

The Node.js layer is designed to handle all I/O bound external API fetching to keep the ML server purely compute-focused.

- **Geocoding:** If only a district/state is provided, the `agroServices.getGeocode` function queries OpenStreetMap (Nominatim) using a 5-strategy fallback system to resolve exact Latitude and Longitude.
- **Predictive Climate Fetching:** `agroServices.getWeather` queries Open-Meteo for a 14-day forecast. Instead of using a single day's transient weather (which causes prediction bias), it calculates the **14-day average temperature** and **cumulative rainfall** to generate a stable, agronomically sound "seasonal baseline."
- **Soil Intelligence:** If the user selects Auto Soil mapping, `agroServices.getSoil` queries the SoilGrids REST API to determine pH, Nitrogen levels, and soil texture at a depth of 0-30cm based on the coordinates.
- **In-Memory Caching:** All of the above external API calls are heavily cached using an LRU mapping (`((lat, lon) -> data)`) with a 1-hour TTL. This means repeated requests for the same region skip the network entirely, resulting in sub-10ms resolution.

### Step 3: Python ML Interception (`app.py` & `soil_mapper.py`)

Node.js compiles a `farmer_inputs` payload and sends it to the Flask server's `POST /predict` endpoint.

- **Feature Engineering:** The `engineer_features_single()` function synthesizes 11 advanced agronomical features from the base inputs. This includes calculating the `np_ratio`, `environmental_stress_score` (penalty for heat/rain extremes), and `fertility_index`.
- **LRU Inference Caching:** The core feature engineering and prediction logic is wrapped in a Python `@lru_cache`. If an identical environmental vector is submitted, it instantly returns the cached prediction without engaging CPU cycles for inference.

### Step 4: Hybrid ML Scoring

AgroXAI uses a dual-engine scoring system to prevent AI hallucinations and ensure realistic recommendations:

1. **XGBoost Probabilities:** The `crop_model.pkl` model outputs raw class probabilities across all trained crops.
2. **Agricultural Compatibility Index (ACI):** A custom algorithm (`compute_aci`) calculates a soft Gaussian penalty for how far the inputs drift from the historical means of each crop (stored in `crop_profiles.pkl`).
3. **Final Ranking:** A `hybrid_score` is calculated as  $0.70 * \text{ML\_Probability} + 0.30 * \text{ACI}$ . This ensures that even if XGBoost is overly confident about an edge-case crop, the ACI bounds the recommendation to historical reality.

### Step 5: Explainable AI & Post-Processing

- **Explanation Matrix:** The Python server maps the XGBoost feature importances against the user's specific environmental limits to generate positive/negative "suitability" metrics for each variable.
- **Final Safety Filter:** The Flask server returns the top crops to Node.js. Node.js runs a final safety check (`filterCrops`) against its strict `CLIMATE_CROP_MAP` to violently reject any crop that might survive the mathematical model but would instantly die in the current physical climate zone.
- **Frontend Render:** The React UI receives the filtered top 3 crops, confidence scores, and the explainability matrix, rendering the premium glassmorphism data panels.

## 3. Key Optimizations

- **Zero-Latency Internal Networking:** Moving geocoding and weather from internal HTTP calls to native service functions (`agroServices.js`) removed over 1000ms of TCP overhead per request.
- **Multi-Layer Caching:** Both the Node.js I/O layer and the Python Compute layer maintain independent caches, scaling the backend efficiently.
- **Algorithmic Stability:** Switching to 14-day weather averages fixed the core issue of "Transient Weather Bias", providing the ML model with a true representation of the local growing season.